learn to use them effectively and without creating a mess.

These combinator functions are useful, but so far you've used them only with synchronous things. Much of your program is asynchronous and uses callbacks. Will these techniques work for asynchronous code?

6.4.5. Asynchronous combinators

Asynchronous functions that accept callbacks are more difficult to compose than functions that are used purely for their evaluation. So far you've looked at using combinators for a synchronous database and for synchronous robots. The real world isn't so kind. Most of your CoffeeScript programming will be asynchronous.

Consider what happens when multiple asynchronous function calls must be called in order. Start with a fake asynchronous function that requires a callback:

```
forward = (callback) ->
  setTimeout callback, 1000
```

You might have seen this written when the function needs to be called five times in sequence:

```
forward ->
  forward ->
    forward ->
      forward ->
        forward ->
          console.log 'done!'
```

Doing this will get painful very quickly.

The nasty-looking cascade you see here is a common affliction in asynchronous programs. Surely there's a way to defeat this problem by composing functions. To simplify things, consider two standalone asynchronous functions, `start` and `forward`:

```
start = (callback) ->
  console.log 'started'
  setTimeout callback, 200
```